IMPLEMENTACION PARA EL MANEJO DE ARCHIVOS Y NAVEGACION SOBRE ELLOS

Análisis para la optimización de recursos

Por qué un arreglo contiguo dinámico y no una lista enlazada doble circular?

- El enunciado exige eficiencia de acceso y memoria. Una lista doblemente enlazada requiere almacenar dos punteros (anterior y siguiente) por cada nodo (16 bytes adicionales por foto en arquitecturas de 64 bits).
- En su lugar, utilizamos un **arreglo contiguo de memoria dinámica (Foto* fotos)** con aritmética modular. Esto garantiza localidad de referencia en caché de la CPU, $O(1)$ de acceso directo y cero sobrecarga de punteros de enlace.

La Lógica Circular (Aritmética Modular):

- Sea N la cantidad de fotos en el álbum (cantidadFotos) y i el índice actual.
- **Avanzar (N o n):** $i_{nuevo} = (i + 1) \pmod N$. Cuando el usuario está en la última foto ($i = N - 1$), $(N - 1 + 1) \pmod N = 0$, saltando limpiamente al inicio.

- **Retroceder (P o p):** $i_{nuevo} = (i - 1 + N) \pmod N$. Sumar N evita obtener índices negativos en C++. Si estamos en la primera foto ($i = 0$), $(0 - 1 + N) \pmod N = N - 1$, regresando al final.

Qué ocurre en memoria cuando interactúas (Acceso por Punteros y Direcciones):

- El atributo fotos es un puntero base que apunta a la primera celda del bloque dinámico (&fotos[0]).
- Cuando se visualiza o modifica la foto en el índice i (por ejemplo, al dar like con 'L'), el compilador no realiza búsquedas secuenciales lentas: calcula la dirección física exacta en la memoria RAM mediante aritmética de punteros:

$$\text{Dirección de la foto} = \text{fotos} + (i \times \text{sizeof(Foto)})$$

- La operación se realiza **in-place** (directamente en la memoria RAM asignada al objeto). No se crea ninguna copia auxiliar en la pila (*stack*), cumpliendo con la directriz de no generar duplicados innecesarios.

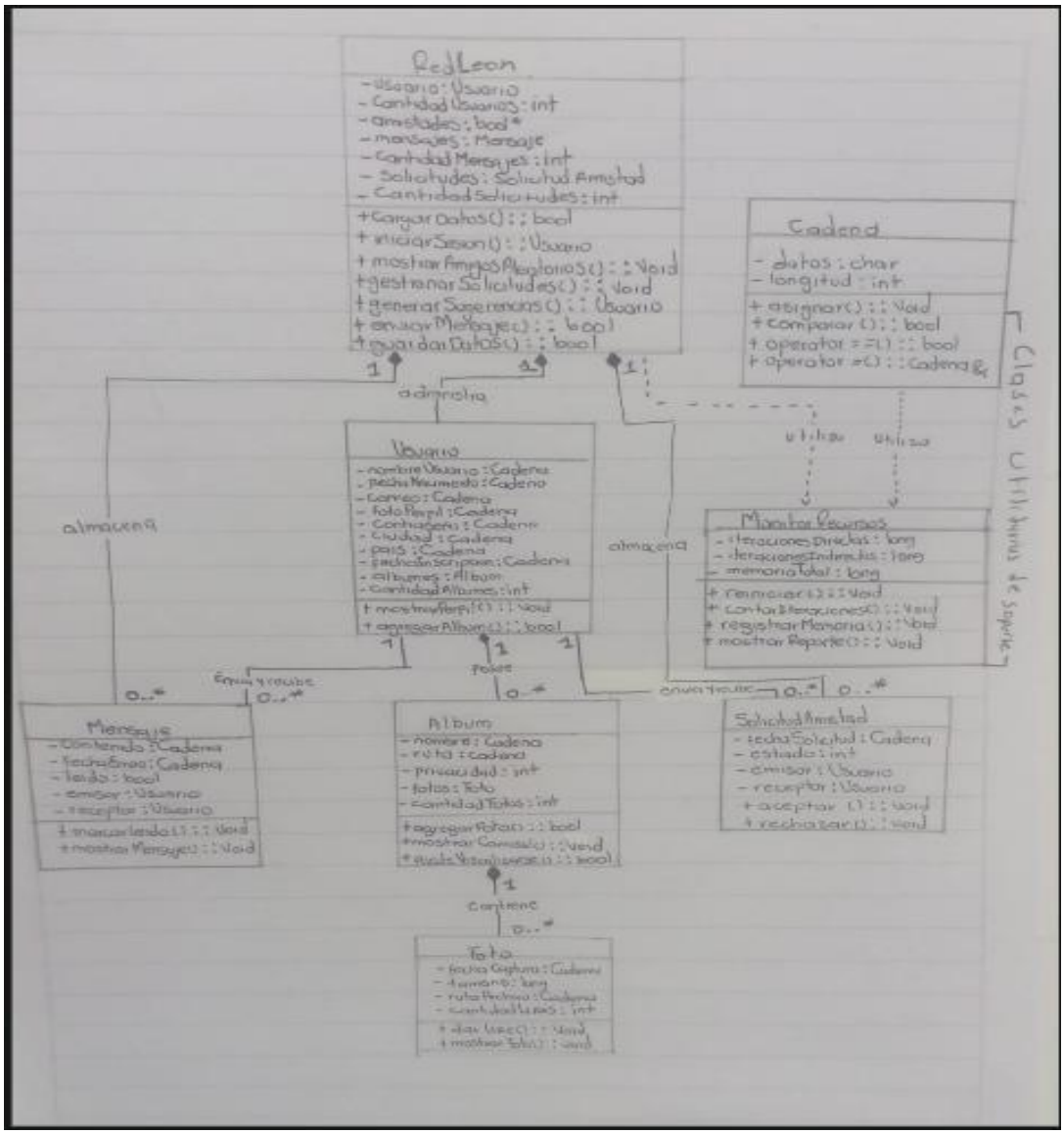
Manejo de archivos planos

1. usuarios.txt
2. amistades.txt
3. albumes_fotos.txt
4. mensajes.txt
5. solicitudes.txt

Nota: Estos son los archivos que **En el código C++:** En la función constructora que genera esos archivos si aún no existen (generarArchivosPruebaSiNoExisten(), dentro de RedLeon.cpp).

La idea de esto es la eficiencia y mantener archivos tanto para memoria temporal en la ejecución como pasar ala memoria permanente los datos de la interacciones y demás que se deben conservar en el perfil.

Diagrama de clases definitivo


RedLeon
- usuario : Usuario
- cantidadUsuarios : int
- amistades : bool *
- mensajes : Mensaje
- cantidadMensajes : int
- Solicitudes : SolicitudAmistad
- CantidadSolicitudes : int
+ CargarDatos() : : bool
+ iniciarSesion() : : Usuario
+ mostrarAmigosRegistrados() : : Void
+ gestionarSolicitudes() : : Void
+ generarSugerencias() : : Usuario
+ enviarMensaje() : : bool
+ guardarDatos() : : bool
Cadena
- datos : char
- longitud : int
+ asignar() : : Void
+ comparar() : : bool
+ operator = () : : bool
+ operator =() : : Cadena &
Clases Utilitarias de Soporte
1
1
1
administra
utiliza
utiliza
Usuario
- nombreUsuario : Cadena
- fechaNacimiento : Cadena
- Correo : Cadena
- FotoPerfil : Cadena
- Contraseña : Cadena
- Ciudad : Cadena
- pais : Cadena
- fechaInscripcion : Cadena
- albumes : Album
- CantidadAlbumes : int
+ mostrarPerfil() : : Void
+ agregarAlbum() : : bool
MonitorRecursos
- iteracionesDirectas : long
- iteracionesIndirectas : long
- memoriaTotal : long
+ reiniciar() : : Void
+ contarIteraciones() : : Void
+ registrarMemoria() : : Void
+ mostrarReporte() : : Void
almacena
almacena
1
1
1
Envia y recibe
0..*
0..*
posee
0..*
envia y recibe
0..*
0..*
Mensaje
- Contenido : Cadena
- fechaEnvio : Cadena
- leido : bool
- emisor : Usuario
- receptor : Usuario
+ marcarLeido() : : Void
+ mostrarMensaje() : : Void
Album
- nombre : Cadena
- ruta : cadena
- privacidad : int
- fotos : Foto
- cantidadFotos : int
+ agregarFoto() : : bool
+ mostrarContenido() : : Void
+ quitarFoto(nombre) : : bool
SolicitudAmistad
- fechaSolicitud : Cadena
- estado : int
- emisor : Usuario
- receptor : Usuario
+ aceptar () : : void
+ rechazar() : : Void
1
contiene
0..*
Foto
- fechaCaptura : Cadena
- tamaño : long
- rutaArchivo : Cadena
- cantidadUsos : int
+ darUso() : : Void
+ mostrarFoto() : : void